

Code Review Report

Language: python | Review mode: hybrid | Model: llama-3.3-70b-versatile | Generated: 2026-08-13 17:40 UTC

Overall Assessment

The code review revealed 12 findings, with 9 critical and 3 high-severity issues. The findings include hardcoded passwords and API keys, SQL injection vulnerabilities, and arbitrary code execution via eval and exec.

Merge readiness: Not Ready

Code health: 0/100

Findings Summary

Total findings: 12

Risk score: 45

Severity breakdown: Low 0, Medium 0, High 3, Critical 9

Type	Severity	Line	Message	Recommendation
Hardcoded Password	High	1	password = "admin123"	Store passwords in environment variables.
Hardcoded API Key	High	3	api_key = "123456789"	Store secrets securely.
SQL Injection	Critical	7	query = "SELECT * FROM users WHERE name='" + name + "'"	Use parameterized queries.
Dangerous Dynamic Execution	Critical	9	eval(input())	Avoid dynamic code execution.
Dangerous Dynamic Execution	Critical	11	exec("print('Hello')")	Avoid dynamic code execution.
Command Injection	Critical	15	subprocess.run("dir", shell=True)	Do not use shell=True with user input.
Security	Critical	1	Hardcoded password	Use environment variables or a secure secrets management system to store sensitive information
Security	Critical	3	Hardcoded API key	Use environment variables or a secure secrets management system to store sensitive information
Security	Critical	5	SQL injection vulnerability	Use parameterized queries or prepared statements to prevent user input from being executed as SQL code
Security	Critical	7	Arbitrary code execution via eval	Avoid using eval and instead use safer alternatives such as ast.literal_eval or json.loads

Type	Severity	Line	Message	Recommendation
Security	Critical	9	Arbitrary code execution via exec	Avoid using exec and instead use safer alternatives such as subprocess.run with a list of arguments
Security	High	11	Use of shell=True with subprocess	Use subprocess.run with a list of arguments instead of shell=True to prevent shell injection attacks

Remediation Roadmap

Finding	Severity	Impact	Suggested Action
SQL Injection	Critical	High	Replace string concatenation with parameterized queries
Dangerous Dynamic Execution	Critical	High	Replace eval and exec with safer alternatives
Command Injection	Critical	High	Replace shell=True with shell=False and pass the command as a list of arguments
Hardcoded Password	High	Medium	Store passwords in environment variables
Hardcoded API Key	High	Medium	Store secrets securely
Use of shell=True with subprocess	High	Medium	Use subprocess.run with a list of arguments instead of shell=True

Source Code Snapshot

```
password = "admin123"

api_key = "123456789"

name = input()

query = "SELECT * FROM users WHERE name='" + name + "'"

eval(input())

exec("print('Hello')")

import subprocess

subprocess.run("dir", shell=True)
```